

Modern Java: From Java 8 to Java 25

A practical, interview-ready guide to records, sealed classes, virtual threads, pattern matching, JPMS, and safe JDK migration.

Audience: a Java 8 developer preparing for professional work, design discussions, or interviews on Java 17/21/25-era code.

How to use this guide

For every topic, learn the intent first, then the boundaries. The strongest professional answer is not 'this feature is newer'; it is 'this feature makes this model safer or this workload simpler, while these constraints still apply.'

Priority	What to master
Must be fluent	Records, sealed types, switch and instanceof patterns, Java 21 virtual threads, migration fundamentals
Must understand	JPMS boundaries, reflection impact, executor and connection-pool limits
Good awareness	Current LTS cadence, preview vs final features, JDK vendor/support choices

1. Records

A record is a concise declaration for a transparent data carrier. It declares final component fields, a canonical constructor, accessor methods named after the components, and value-oriented implementations of `equals`, `hashCode`, and `toString`.

Use a record when the primary purpose of a type is to carry a small, stable set of values. A DTO at an API boundary, a query result, a command object, a value object, an event, or a configuration snapshot are typical fits.

Java 8 comparison

```
// Java 8: boilerplate obscures the value semantics
public final class Money {
    private final BigDecimal amount;
    private final Currency currency;
    // constructor, getters, equals, hashCode, toString...
}

// Modern Java
public record Money(BigDecimal amount, Currency currency) {}
```

What is generated and what is not

Property	Record behavior	Practical implication
Fields	Private and final	The field reference cannot be reassigned after construction.
Accessors	<code>amount()</code> , not <code>getAmount()</code>	Frameworks may need record support or configuration.
Equality	Component-by-component	Ideal for value objects; do not use identity-based expectations.
Inheritance	Implicitly final; may implement interfaces	Do not use records for an extensible base class.
Constructors	Canonical/compact constructors allowed	Validate invariants at creation time.

Use when

- The data model is complete at construction and represents a value.
- You want meaningful value equality, logging, and less boilerplate.
- You can express validation in the canonical or compact constructor.
- The type should not be subclassed.

Avoid or reconsider when

- The object has a mutable lifecycle, complex invariants that evolve after construction, or behavior is its main purpose.
- A persistence framework requires a no-argument constructor, field mutation, or proxy subclassing. Modern JPA support varies by provider/version; verify it rather than assume.
- You need identity semantics: two different database entities with the same fields may not be interchangeable.
- You need deep immutability but components include arrays, lists, maps, dates, or mutable domain objects.

Immutability: the critical caveat

Records are shallowly immutable. Their component references are final, but the referenced objects may still change. A record containing a mutable list exposes the same mutable list unless you make a defensive copy.

```
// Unsafe: callers can mutate the list after record creation.
record Order(String id, List<String> items) {}

// Safer: snapshot at construction. List.copyOf also rejects null elements.
record Order(String id, List<String> items) {
    Order { items = List.copyOf(items); }
}
```

For arrays, copy on both input and output; a final array reference is still mutable.

```
record Digest(byte[] bytes) {
    Digest { bytes = bytes.clone(); }
    @Override public byte[] bytes() { return bytes.clone(); }
}
```

Interview question: Are records immutable?

They are shallowly immutable: component fields are final, but an object referenced by a component can still mutate. Use immutable component types or defensive copying to get deep immutability.

Interview question: Can a record declare methods and implement an interface?

Yes. Records can contain methods, static fields, and validation, and can implement interfaces. They cannot extend another class because they already extend `java.lang.Record`.

Interview question: Should every DTO be a record?

No. Use one when its value-carrier semantics match. Framework binding, persistence, binary compatibility, mutable patch payloads, or domain behavior may favor a class.

2. Sealed classes and interfaces

A sealed type explicitly controls which classes may directly extend or implement it. This turns a previously open hierarchy into a closed, compiler-known domain.

Use sealed types when your domain has a finite, intentional set of alternatives: payment outcomes, parser tokens, authorization decisions, AST nodes, or internal state machines.

Java 8 comparison

```
// Java 8: anyone can add another subtype; a switch needs default handling.
interface Payment {}

// Java 17+: the valid alternatives are explicit.
sealed interface Payment permits CardPayment, UpiPayment, CashPayment {}
record CardPayment(String last4) implements Payment {}
record UpiPayment(String address) implements Payment {}
record CashPayment() implements Payment {}
```

Permitted direct subclasses must be final, sealed, or non-sealed. non-sealed deliberately reopens that branch.

Exhaustive switch

```
static String receiptLabel(Payment payment) {
    return switch (payment) {
        case CardPayment card -> "Card ending " + card.last4();
        case UpiPayment upi    -> "UPI " + upi.address();
        case CashPayment cash  -> "Cash";
    };
} // No default needed: compiler knows all permitted cases.
```

Why this matters

- Adding a new permitted subtype forces relevant exhaustive switches to be reconsidered at compile time.
- The type system records a business constraint instead of relying on documentation and a fragile default branch.
- Records and sealed interfaces combine well for immutable algebraic-data-type style models.

Use when

- The set of legal states is controlled by your team/module.
- A default branch would hide an unhandled business case.
- The hierarchy is a domain model, not an extension point for third parties.

Avoid or reconsider when

- You publish a library specifically designed for third-party extension.
- Plugins or user-defined implementations are the core product requirement.
- A plain enum is sufficient. Use enum for a fixed list of constants without distinct per-variant data/behavior.

Choose	When
enum	Fixed constants, usually singleton-like values; limited variant-specific data.
sealed interface + records	Finite variants with different data shapes and behavior.
ordinary interface/abstract class	Intentional external extension or an open plugin ecosystem.

Interview question: Does sealed mean an object cannot be extended at runtime?

It means direct subtypes are restricted by the compiler and JVM rules. The permitted list is part of the type declaration. It is about hierarchy control, not making object state immutable.

Interview question: Why avoid default in an exhaustive switch?

A default can suppress compiler help when a new case is added. Omit it when all known cases are deliberately handled; use default only when the domain really has an intentionally catch-all rule.

3. Virtual threads

Virtual threads are lightweight JVM-managed threads designed to make high-concurrency, blocking I/O code scalable without writing continuation-heavy asynchronous code. They are final in Java 21.

Use them for many concurrent tasks that mostly wait: HTTP calls, JDBC calls, file/network I/O, message handling, and request fan-out. They keep the familiar sequential programming model.

Java 8 comparison

```
// Java 8 platform-thread pool: pool size limits concurrent blocking calls.
ExecutorService pool = Executors.newFixedThreadPool(200);

// Java 21: create one virtual thread per task; do not pool virtual threads.
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<Response> response = executor.submit(() -> client.call(request));
}
```

What virtual threads do not solve

Constraint	Why virtual threads do not remove it	What to do
CPU	CPU work still needs CPU cores.	Bound CPU-parallel work; profile and tune algorithms.
Database connections	Each blocking query can still consume a limited physical connection.	Keep pool bounds, apply backpressure, size by DB capacity.
Remote API quota	More callers can overload a downstream service faster.	Use timeouts, concurrency limits, retries with backoff, rate limits.
Memory	Millions of tasks still have stacks, request data, and queues.	Bound work and observe heap/queue growth.

When to use

- A service is thread-pool constrained because requests mostly wait on I/O.
- Existing synchronous code is correct and you want simpler high-concurrency scaling.
- You can set timeouts/cancellation and preserve downstream bulkheads.

When not to use blindly

- CPU-intensive tasks. A virtual thread per CPU-heavy job does not create more cores.
- As a reason to delete existing database-pool limits or remote-service throttles.
- As an unbounded task queue. A cheap thread is not a free request.
- Where libraries rely on long synchronized blocking sections or thread-local-heavy designs; test and profile.

Pinning and blocking

When a virtual thread blocks in a synchronized region or native code, it can pin its carrier thread. Modern JDKs have improved monitor behavior, but long blocking operations under synchronized merit investigation. Prefer short critical sections and use `ReentrantLock` where its semantics are appropriate. Diagnose with JFR and virtual-thread pinning events.

```
// Good boundary: virtual threads handle request concurrency;
// a semaphore protects a bounded downstream resource.
Semaphore dbSlots = new Semaphore(40);
executor.submit(() -> {
    dbSlots.acquire();
```

```
    try { return repository.read(id); }  
    finally { dbSlots.release(); }  
  });
```

Interview question: Should I pool virtual threads?

Usually no. The executor is intentionally thread-per-task. Pool scarce resources such as connections or API concurrency, not virtual threads.

Interview question: Are virtual threads faster?

They improve scalability and resource efficiency for blocking I/O. Individual CPU work is not inherently faster; benchmark the actual workload.

Interview question: Can I replace reactive programming everywhere?

Not automatically. Virtual threads simplify many request/response services, but reactive streams can still be useful for stream processing, explicit backpressure, and ecosystems already built around them.

4. Pattern matching

Pattern matching lets Java test a value's type and bind a correctly typed variable in one construct. It removes repeated casts and makes intent more explicit. Pattern matching for `instanceof` is final in Java 16; pattern matching for `switch` and record patterns are final in Java 21.

Use it where a value may have one of several known shapes and behavior genuinely depends on that shape. Combined with sealed types, it makes domain branching concise and checked.

Java 8 comparison

```
// Java 8
if (value instanceof String) {
    String text = (String) value;
    return text.isBlank();
}

// Modern Java
if (value instanceof String text) {
    return text.isBlank();
}

// Record patterns: deconstruct a nested record directly.
record Point(int x, int y) {}

String describe(Object value) {
    return switch (value) {
        case Point(int x, int y) when x == y -> "diagonal";
        case Point(int x, int y)             -> "(" + x + ", " + y + ")";
        case null                             -> "missing";
        default                               -> "other";
    };
}
```

Use when

- Type dispatch is legitimate and localized, such as parsing an AST, processing events, or translating a sealed domain model.
- The condition can be made safer by binding a typed variable only after its type is proven.
- A switch expression can return a value and be exhaustive.

Avoid or reconsider when

- You are switching on types because a missing polymorphic method would be clearer.
- The hierarchy is open and the default branch would quietly absorb future types.
- The condition is too complex; move business rules into named methods.

Important semantics

Topic	Must know
Scope	A pattern variable exists only where Java can prove the type matched. With <code>&&</code> , it is in scope on the right side; with <code> </code> , it generally is not.
Null	<code>instanceof</code> returns false for null. A pattern switch throws NPE for null unless it has case <code>null</code> .
Guards	Use <code>when</code> for a guarded pattern case. Put the more specific guarded case before the general one.

Topic	Must know
Dominance	A broad case before a narrower case is a compile-time error because the narrow case is unreachable.

Interview question: Is pattern matching a replacement for polymorphism?

No. Prefer polymorphism when behavior belongs to each subtype and the hierarchy is open or behavior is distributed. Pattern switches excel at centralized interpretation of a finite sealed model.

Interview question: Why is case ordering important?

Cases are checked for dominance. A broad type pattern may make a later narrower pattern unreachable. Put precise/guarded cases before general ones.

5. Java Platform Module System (JPMS)

JPMS, introduced in Java 9, adds reliable configuration and strong encapsulation to Java. A named module declares what it requires and what packages it exports. The module path is different from the traditional classpath.

Use JPMS deliberately when you control a modular application or library and want enforceable boundaries. It is valuable for large platforms and internal libraries, but adoption has costs in reflection-heavy and legacy ecosystems.

Basic module descriptor

```
// src/com.example.billing/module-info.java
module com.example.billing {
    requires java.net.http;
    exports com.example.billing.api;
    // com.example.billing.internal is NOT exported
}
```

Classpath (Java 8 style)	Module path (JPMS)
A JAR can generally access public classes in other JARs.	A module must read the target module, and the target package must be exported.
Dependency failures often appear at runtime.	Missing/illegal dependencies are more likely to fail at compile/startup time.
Public often means broadly accessible.	Public types in non-exported packages are strongly encapsulated.

Why many projects remain on the classpath

- Frameworks often use reflection, bytecode generation, proxies, or classpath scanning. These can need explicit opens directives.
- Dependencies may be automatic modules or have unstable module names.
- A multi-module build adds design and tooling complexity without enough benefit for a small deployable service.
- Migration may expose accidental dependencies on JDK internals or split packages.

exports vs opens

Directive	Allows	Typical use
exports package	Compile-time and normal runtime access to public types	A supported API package for consumers.
opens package	Deep reflection at runtime	JPA/Jackson/Spring-like reflective access; often qualified to a specific module.
requires transitive	Consumers also read the dependency	Use only when the dependency's types appear in your public API.

Practical migration posture

Do not make 'add module-info.java' the first goal of a Java 8 upgrade. First run the app on a supported modern JDK on the classpath, resolve dependency and reflection problems, then modularize if boundary enforcement provides enough value.

```
// Qualify reflective access rather than opening everything.
module com.example.app {
    requires com.fasterxml.jackson.databind;
    opens com.example.app.dto to com.fasterxml.jackson.databind;
}
```

}

Interview question: Does module-info.java replace Maven or Gradle?

No. Maven/Gradle manage builds and dependency resolution. JPMS is a Java-language/runtime dependency and encapsulation system. They solve related but different problems.

Interview question: What is a split package and why is it a problem?

A split package is the same package spread across multiple modules. JPMS rejects it because it makes ownership and access ambiguous. Repackage or redesign the modules.

Interview question: Should every Spring Boot service use JPMS?

Not necessarily. It can work, but classpath deployment is often the pragmatic choice. Adopt JPMS where enforceable boundaries and runtime images outweigh integration cost.

6. Migration from Java 8

A safe migration is a program of compatibility validation, not merely changing `sourceCompatibility`. Separate language/source compatibility, binary compatibility, runtime behavior, dependencies, build plugins, containers, security, and operational tooling.

A practical target is Java 17 or Java 21 for existing systems. Java 25 is the current LTS and is appropriate when your organization is ready for it. Choose the supported baseline required by your platform, vendors, and security policy.

Compatibility dimensions

Dimension	Question	Typical failure
Source	Will the code compile with the new compiler?	Stricter compiler checks, removed internal APIs.
Binary	Can existing compiled libraries link?	NoSuchMethodError, incompatible bytecode/library versions.
Runtime	Does behavior remain correct on the newer JDK?	Reflection access failures, changed TLS/defaults, locale/timezone differences.
Build/tooling	Are Maven/Gradle plugins, annotation processors, tests, and IDE settings compatible?	Old bytecode plugin, obsolete test runner, compiler plugin mismatch.
Operations	Do container images, agents, monitoring, and JVM flags work?	Unrecognized VM option, unsupported agent, base image mismatch.

Recommended migration sequence

Step	Action	Evidence of success
1. Inventory	Record current JDK, vendor, dependencies, plugins, JVM flags, agents, base image, and external integrations.	Versioned inventory and ownership.
2. Build on target JDK	Compile and run tests without changing language level first.	Repeatable CI build on target JDK.
3. Upgrade dependencies	Use versions explicitly supporting the target JDK; update frameworks/plugins/processors.	No unsupported/transitive legacy libraries.
4. Fix runtime access	Test reflection, serialization, TLS, logging, and startup paths.	Clean startup without broad <code>--add-opens</code> workarounds.
5. Performance/load test	Compare latency, throughput, CPU, heap, GC, thread count, error rate.	Measured baseline and no regression.
6. Roll out safely	Canary, rollback plan, dashboards, supported JDK distribution/patch policy.	Observable controlled production release.

Removed and obsolete assumptions

- Java EE/CORBA modules were removed from the JDK. Use external dependencies such as Jakarta APIs where needed.
- Nashorn was removed; do not assume an embedded JavaScript engine exists.
- JDK internal APIs are strongly encapsulated. Replace `sun.*` usage with supported APIs.

- CMS GC was removed. Re-evaluate old garbage-collector flags; modern defaults and G1/ZGC options differ by JDK.
- Do not carry old --add-opens flags indefinitely. They are temporary compatibility bridges, not a design solution.

Testing that catches real migration defects

- Run unit, integration, contract, and end-to-end tests on the target runtime - not only compilation.
- Exercise TLS/mTLS, JDBC, serialization, reflection/proxies, timezone/locale, file encoding, and container startup.
- Run load tests long enough to observe GC, connection pools, thread behavior, and downstream throttling.
- Test the exact production JDK vendor, patch level, container base image, and runtime flags.

JDK distribution and support

OpenJDK builds and commercial distributions differ in support, patch cadence, licensing, and operational policy. Select a supported distribution deliberately (for example, your organization's approved vendor), keep it patched, and document the update cadence. LTS is a support/lifecycle decision; it does not make an upgrade risk-free.

Interview question: Can I compile with Java 17 and still run on Java 8?

Only if you deliberately target Java 8 bytecode and APIs, for example with --release 8, and do not use newer language/runtime features. In practice, choose and test the runtime you deploy.

Interview question: Why did code compile but fail with InaccessibleObjectException?

Compilation checks source APIs; runtime reflection may require access to non-exported packages under strong encapsulation. Upgrade the library first; use narrow opens only where justified.

Interview question: Is changing the JDK enough to gain virtual-thread benefits?

No. You need Java 21+ and intentional concurrency design. Measure blocking paths and retain limits around databases/downstream services.

Interview and Design Review Cheat Sheet

Topic	One-sentence professional answer
Records	Use for value-oriented data carriers; remember they are shallowly immutable and require defensive copies for mutable components.
Sealed types	Use them to model a closed set of valid alternatives and let the compiler enforce exhaustive handling.
Virtual threads	Use for high-concurrency blocking I/O, while still bounding scarce resources and CPU-heavy work.
Patterns	Use patterns to make finite type dispatch safer and clearer; do not replace good polymorphism with a giant switch.
Modules	JPMS gives reliable dependencies and encapsulation, but classpath remains pragmatic in reflection-heavy or legacy applications.
Migration	Treat it as a compatibility and operations project: compile, dependency, runtime, performance, and rollout validation all matter.

Frequently tested traps

Interview question: A record has final fields, so it is deeply immutable - correct?

Incorrect. Final prevents reference reassignment, not mutation through that reference.

Interview question: Virtual threads remove the need for a JDBC pool - correct?

Incorrect. They allow more blocked callers; the database still has a finite safe connection/concurrency limit.

Interview question: A sealed hierarchy is always better than an ordinary interface - correct?

Incorrect. It is better only for a deliberately closed domain. Open extension points should remain open.

Interview question: Pattern switch always handles null - correct?

Incorrect. Include case null if null is an allowed input; otherwise it can throw NPE.

Interview question: JPMS requires all Java applications to stop using the classpath - correct?

Incorrect. Named modules are optional; classpath applications remain supported and often practical.

Interview question: A successful compile proves a Java 8 upgrade is safe - correct?

Incorrect. Runtime reflection, dependencies, agents, flags, TLS, and behavior require testing.

Suggested learning order

1) Switch expressions/text blocks/var and collection factories. 2) Records and sealed interfaces together. 3) instanceof patterns, record patterns, and exhaustive switch. 4) Virtual threads with real I/O and bounded resources. 5) Java 8-to-LTS migration labs. 6) JPMS after understanding module-path tradeoffs.

Version note

The practical LTS baseline for many organizations remains Java 17 or Java 21. Java 25 is the current LTS. Always align with the organization's approved JDK vendor, framework support matrix, and patch policy.

End of guide